

## Eksperyment 4.5, Faza 5 – Embeddingi neuronowe zamiast ręcznych cech

Czy gotowy model embeddingu tekstu (bez treningu, bez wiedzy o CAN) bije 7-wymiarowy, ręcznie zaprojektowany wektor cech z Eksperymentu 4.4? I czy w ogóle da się to uruchomić na Raspberry Pi Zero W?

### 1. NAJPIERW SPRZĘT – SPRAWDZONE PRZED ANALIZĄ JAKOŚCI

#### TWARDY BLOKIER: ARMV6

Sprawdzone bezpośrednio na Pi Zero W (nie założone z góry):

PyTorch

✗ brak pakietu (piwheels.org)

ONNX Runtime

✗ brak pakietu

TensorFlow Lite

✗ brak pakietu

To nie kwestia wydajności – żaden mainstream framework ML nie ma w ogóle zbudowanych pakietów dla tej architektury. Ekosystem ML porzucił ARMv6 lata temu.

#### ŚCIEŻKA DALSZĄ: ORANGE PI ZERO 3

**PyTorch ma oficjalne koło** `manylinux2014_aarch64` na PyPI (pobrane, 91.9MB) – Orange Pi Zero 3 (Cortex-A53, ARMv8/aarch64) to generacyjnie inny procesor niż ARM11 w Pi Zero W. Do potwierdzenia na samej płytce (czy system jest 64-bit) + inny moduł CAN (inne złącze GPIO niż Pi).

### 2. METODYKA JAKOŚCI (OFFLINE, LAPTOP X86\_64) – TRZY TESTY, ROSNĄCO RYGORYSTYCZNE

#### MODEL I SERIALIZACJA

**all-MiniLM-L6-v2** (sentence-transformers, embedding 384-wym.) – model wytrenowany na języku naturalnym, NIGDY na sekwencjach liczbowych. Seria wartości bajtu/bitu zamieniana na tekst (liczby oddzielone spacją), embedowana bezpośrednio – celowo "out of distribution" zastosowanie, testowane empirycznie bez zakładania z góry że zadziała.

3. WYNIKI – CECHY RĘCZNE VS EMBEDDING NEURONOWY

Test A – w obrębie jednego korpusu (leave-one-out)



Test B – cross-corpus (offline ↔ offline)



Test C – cross-corpus (offline ↔ REALNE dane z magistrali, N=162)



■ cechy ręczne (7-dim, Eksperyment 4.4)    ■ embedding neuronowy (384-dim, all-MiniLM-L6-v2)

| TEST                           | CECHY RĘCZNE | EMBEDDING NEURONOWY | Δ      |
|--------------------------------|--------------|---------------------|--------|
| A – w obrębie korpusu          | 89.5%        | 98.7%               | +9.2pp |
| B – cross-corpus (offline)     | 92.8%        | 100.0%              | +7.2pp |
| C – cross-corpus (realne dane) | 89.5%        | 98.8%               | +9.3pp |

Wartość bazowa cech ręcznych w Teście C (89.5%) jest wyższa niż "~70%" cytowane w oryginalnym raporcie 4.4 – to INNY protokół ewaluacji (3-way leave-one-out/cross-query tutaj, vs trafność pojedynczej odpowiedzi w pełnym pipeline LLM tam), nie identyczny pomiar. To, co jest w pełni kontrolowane i porównywalne: cechy ręczne vs embedding NA TYCH SAMYCH danych i metodzie – tu przewaga embeddingu jest spójna i duża we wszystkich trzech testach.

##### JAK JA TO WIDZĘ

**Gotowy model spoza domeny pobił dedykowaną inżynierię cech** — model nie ma pojęcia o "odległości liczbowej" między tokenami takimi jak "45" i "46", a mimo to złapał strukturę lepiej niż heurystyka zaprojektowana specjalnie do tego zadania. Podejrzewam (niepotwierdzone), że embedding tekstowy zachowuje lokalne wzorce KOLEJNOŚCI sekwencji, których 7-wymiarowy wektor ręczny (uśredniający całą serię do pojedynczych liczb) się pozbywa.

**Bariera jest ekosystemowa, nie architekuralna** — w przeciwieństwie do wcześniejszych przeszkód tej sesji (serwer Qdrant, moc obliczeniowa), tej nie da się obejść inaczej niż zmianą sprzętu. Orange Pi Zero 3 (już posiadany) to zweryfikowana częściowo, obiecująca ścieżka.

**Druga faza tej sesji o charakterze rozwiązania, nie tylko diagnozy** — mimo negatywnego wyniku sprzętowego, sam wynik jakościowy jest jednoznaczny, a ścieżka dalsza (Orange Pi) skonkretyzowana, nie zostawiona jako otwarte pytanie.

---

MagistralaCAN4 — esp\_experiment\_4\_5\_rpi/neural\_embedding\_warmstart.py — wygenerowano automatycznie na podstawie wyników bieżącej sesji